

Spring6-Spring Boot3

Lab02

Spring Framework Core

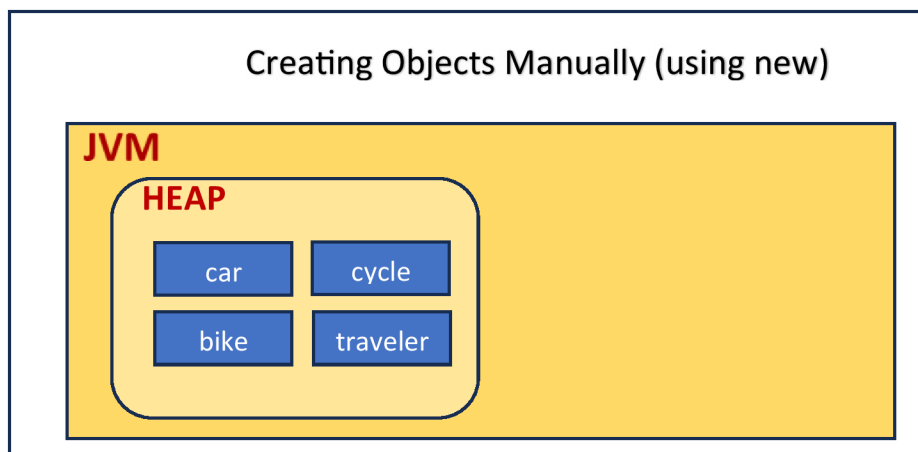
Spring IoC Container

Java Based Configuration

Dans le Lab précédent, nous avons vu ce qu'est un couplage fort et un couplage faible. Et nous avons également vu que nous avons créé une instance de différentes classes en utilisant l'opérateur new.

Lorsqu'on crée des instances de classe manuellement en utilisant l'opérateur new, ces objets sont stockés directement dans la mémoire Heap de la JVM.

Dans une JVM, nous avons une mémoire Heap (ou tas), et chaque fois que nous créons une instance de classes en utilisant l'opérateur new, ces instances seront stockées dans cette mémoire Heap de la JVM.



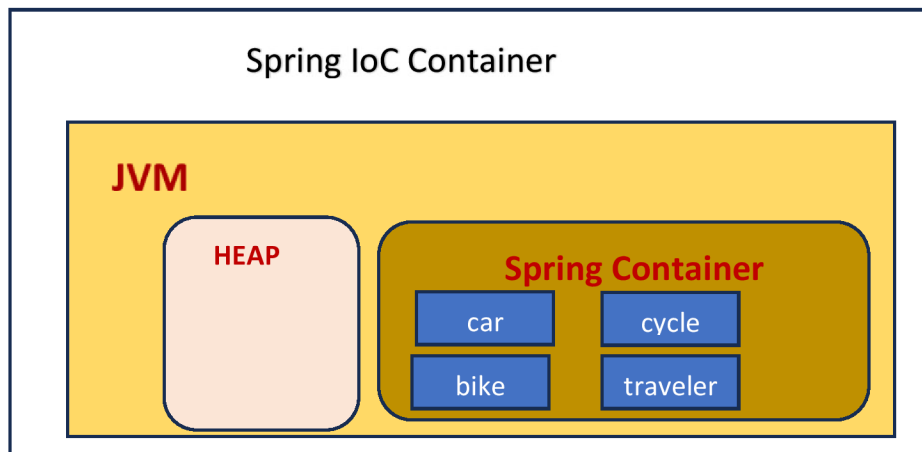
Dans un projet réel, nous pouvons avoir des centaines ou des milliers de classes et nous devons peut-être créer manuellement des instances de ces classes en utilisant l'opérateur new.

Mais ce n'est pas une bonne idée ou une bonne pratique de devoir créer manuellement des instances de centaines de milliers de classes dans un projet.

C'est donc là que Spring Framework entre en scène et fournit la fonctionnalité « **Spring Container** » qui crée essentiellement les objets et gère le cycle de vie de ces objets.

Voyons maintenant ce qu'est Spring IoC Container. IoC signifie Inversion of Control.

Fondamentalement, nous donnons le contrôle à Spring Framework pour créer les objets et gérer leur cycle de vie pour nous. Ainsi nous n'aurons pas à créer les objets manuellement.



Spring IoC Container

1. Responsible for creating the objects (Spring beans)
2. Responsible for injecting one object into another object (using DI)
3. Managing the bean's entire life-cycle - It manages the complete lifecycle of an bean from creation to destruction
4. Container uses the information provided in the configuration file (or annotations) to create the objects, and it uses dependency injection to supply the objects with their dependencies.
5. The configuration metadata is represented in XML, Java annotations, or Java code.
6. By using the IoC container, the objects are more loosely coupled and easier to test, since their dependencies can be easily swapped out with mock objects.

Spring Container est essentiellement responsable de la création des objets que nous désignons par « **spring beans** ».

Qu'est ce qu'un spring bean?

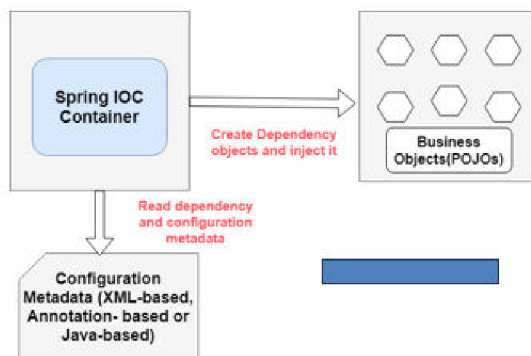
Un **Spring Bean** n'est rien d'autre qu'un objet Java créé et géré par Spring Container. Spring Container est chargé d'injecter un objet dans un autre objet en utilisant l'injection de dépendances. Nous en apprendrons davantage sur ce qu'est l'injection de dépendances, les différents types d'injection de dépendances dans le prochain Lab.

Spring Container gère les beans et leur cycle de vie. Il gère essentiellement le cycle de vie complet du bean, de la création à la destruction. Cela signifie que Spring Container crée les objets et les détruit également chaque fois qu'il est arrêté.

Spring Container utilise les informations fournies dans un fichier de configuration pour créer les objets et utilise l'injection de dépendances pour fournir à ces objets leurs dépendances.

Dans le fichier de configuration, nous devons fournir des informations au Spring container, pour lui indiquer quels sont les objets qu'il doit créer et comment il peut injecter un objet dans un autre objet pour établir la dépendance.

Spring IoC Container



Nous pouvons configurer ces métadonnées qui indiquent au Spring Container quels sont les objets à créer et les injections de dépendances à établir, en utilisant une configuration basée sur un fichier XML, ou une configuration basée sur des annotations ou une configuration basée sur du code Java.

En utilisant Spring Container, les objets sont plus faiblement couplés et plus faciles à tester puisque leurs dépendances peuvent être facilement remplacées par des objets fictifs.

Puisque Spring Container utilise l'injection de dépendances pour injecter des dépendances, les objets sont donc faiblement couplés et plus faciles à tester.

Spring Framework fournit deux types de spring containers :

- Bean factory container
- Application context container

Spring IoC Container Types

1. BeanFactory container
2. ApplicationContext container

BeanFactory Container est responsable de la création des spring beans, de leur configuration et de la gestion de leur cycle de vie.

BeanFactory Container

1. Responsible for creating the beans, configuring the beans and managing the bean's entire life-cycle.

ApplicationContext Container

1. Responsible for creating the beans, configuring the beans and managing the bean's entire life-cycle.

2. Enterprise Application Features:

- It provides messaging (i18n or internationalization) functionality
- Event publication functionality
- Annotation-based dependency injection
- Easy integration with Spring AOP features
- Supports almost all types of bean scopes

ApplicationContext Container est un conteneur avancé. Il fournit des fonctionnalités de niveau application d'entreprise.

En plus de la fonction de base d'un spring container qui consiste à créer les Spring Beans, de les configurer et de gérer leur cycle de vie, Application Context fournit des fonctionnalités d'entreprise telles que :

- La messagerie.
- L'internationalisation
- La publication des événements ;
- L'injection des dépendances basée sur les annotations
- Une intégration facile avec Spring AOP (Aspect Object Programming).

Steps for Java based configuration

1. Create Configuration class with @Configuration annotation
2. Create method and annotated it with @Bean annotation
3. Create Spring IoC Container (ApplicationContext) and Retrieve Spring bean from Spring IoC container.

Dans ce Lab, nous allons voir un exemple d'utilisation de **ApplicationContext** Container avec une configuration basé sur le **code Java**.

Il est important de comprendre comment créer une configuration basée sur Java dans une application Spring.

Donc, avant de comprendre comment utiliser Spring ApplicationContext Container avec une configuration basée sur Java, comprenons étape par étape comment créer une configuration Java dans une application Spring.

Etapes simples pour une configuration basée sur du code Java.

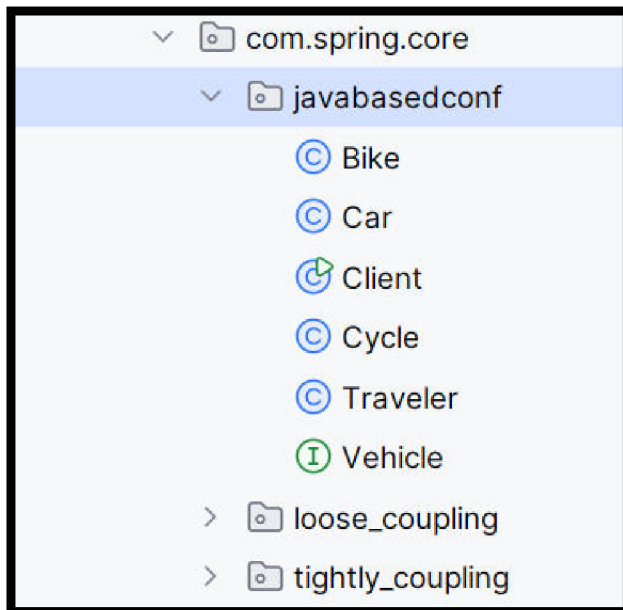
Créez d'abord une classe de configuration avec l'annotation **@Configuration**.

1. Commencer par créer une classe de configuration avec l'annotation **@Configuration**. Une fois que nous annotons une classe avec l'annotation @Configuration, cette classe devient une configuration basée sur du code Java et au sein de cette classe, nous pouvons définir des Spring Beans.
2. Ensuite, afin de créer un spring bean, nous devons créer une méthode et l'annoter avec l'annotation **@Bean**.
3. Ensuite, nous créons un Spring Container qui est un objet **ApplicationContext** qui va nous permettre de récupérer les Spring Beans.

Dans ce qui suit nous présentons un exemple pour présenter ces différentes étapes.

Dans le projet que nous avons créé dans le Lab précédent, commençons par créer un package nommé **com.spring.core.javabasedconf**

Copions dans ce package les classes **Car**, **Bike**, **Cycle**, **Traveler**, **Client** et l'interface **Vehicle** que nous avons créés au Lab N°1 et qui se trouvent dans le package **com.spring.core.loose_coupling**.



Dans une 1^{ère} étape, créons ensuite la classe de configuration avec l'annotation **@Configuration**. Par exemple, nous pouvons créer une classe de configuration nommée **AppConfig**. Ainsi, une fois que nous annotons une classe avec l'annotation **@Configuration**, cette classe devient une classe de configuration et nous pouvons configurer la définition des beans Spring dans cette classe de configuration.

En seconde étape, créons des **méthodes** en les annotant avec l'annotation **@Bean**.

Le code ci-dessous crée les spring beans pour les classes Car, Bike, Cycle et Traveler.

Classe AppConfig.java

```
package com.spring.core.javabasedconf;

//import com.spring.core.javabasedconf.Vehicle;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {
    @Bean
    public Vehicle car(){
        return new Car();
    }

    @Bean
    public Vehicle bike(){
        return new Bike();
    }

    @Bean
    public Vehicle cycle(){
        return new Cycle();
    }
}
```

```

    //--Dependency Injection--
    @Bean
    public Traveler traveler() {
        return new Traveler(bike()); //DI
    }
}

```

Pour créer un spring bean pour la classe Bike par exemple, nous créons la méthode **bike()** qui retourne un objet instance de la classe Bike. Cette méthode **bike()** est annotée par l'annotation **@Bean**.

L'annotation **@Bean** spécifie à Spring Container de gérer l'objet instance de la classe Bike retourné par la méthode **bike()**. Remarquer que l'objet retourné par la méthode **bike()** a été créé manuellement par l'opérateur new. Mais nous aurons une seule création de l'objet avec new. Et chaque fois que nous avons besoin de cet objet dans notre projet, nous pouvons l'obtenir à partir du Spring. Nous n'avons pas besoin de recréer l'objet de cette classe Bike en utilisant une nouvelle fois l'opérateur new.

Ensuite, dans une configuration basée sur Java, nous devons injecter les dépendances manuellement. Par exemple, ici la classe **Traveler** dépend de la classe de **Bike**. Et nous devons transmettre explicitement l'objet de la classe **Bike** manuellement à l'objet de la classe **Traveler** en passant au constructeur l'appel de la méthode bike() qui retourne un objet Bike(**return new Traveler(bike())**).

N'oubliez pas que dans une configuration basée sur Java, nous devons créer manuellement l'objet de la classe, puis @Bean dit au conteneur Spring de commencer à gérer cet objet de la classe. Ensuite, nous devons injecter les dépendances manuellement.

Après, il faut créer un objet spring container et retrouver les spring beans à partir de ce container.

L'interface ApplicationContext agit comme un Spring Container. Donc nous devons créer un objet ApplicationContext et lui passer comme entrée la classe de configuration AppConfig.

Modifions la classe Client pour créer un objet ApplicationContext et l'utiliser pour obtenir les différents Beans.

Classe Client.java

```

package com.spring.core.javabasedconf;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Client {
    public static void main(String[] args) {
        // Creating Spring IOC Container
        // Read the configuration class
        // Create and manage the Spring beans
        ApplicationContext applicationContext = new
        AnnotationConfigApplicationContext(AppConfig.class);
    }
}

```

```
// Retrieve Spring Beans from Spring IOC Container
Car car = applicationContext.getBean(Car.class);
car.move();

Bike bike = applicationContext.getBean(Bike.class);
bike.move();

Cycle cycle = applicationContext.getBean(Cycle.class);
cycle.move();

Traveler traveler =
applicationContext.getBean(Traveler.class);
    traveler.startJourney();
}
}
```

L'objet `ApplicationContext` n'est rien d'autre que `SpringContainer`. Il lira le fichier de configuration qu'on lui a passé comme entrée. Et sur la base de la configuration que nous avons définie dans notre fichier `AppConfig`, il créera des beans spring puis gèrera ces beans spring.

Ensuite, une fois que l'objet `ApplicationContext` est créé, nous pouvons l'utiliser pour récupérer ces beans en utilisant la méthode `getBean()`.

Par exemple pour obtenir l'objet spring bean de la classe `Car`, nous utiliserons l'instruction `applicationContext.getBean(Car.class);`

Si vous observez de près cette configuration basée sur le code Java, nous créons essentiellement l'objet de la classe manuellement, puis nous disons au conteneur Spring de gérer cet objet pour nous.

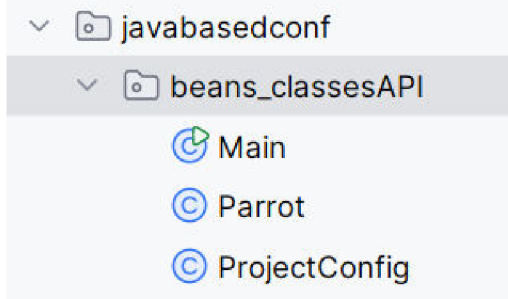
Nous devons créer l'objet une seule fois, puis nous stockerons cet objet dans le conteneur Spring. Ensuite, chaque fois que cela est nécessaire, nous pouvons simplement récupérer cet objet dans le conteneur Spring. Nous n'avons pas besoin de recréer l'objet manuellement en utilisant le mot-clé `new`.

Pour éviter de mélanger les concepts, il est recommandé de mettre toutes les classes dans un nouveau package avec la classe `AppConfig`.

Définition de beans de classes prédéfinies dans l'API Java

Vous pouvez ajouter n'importe quel type d'objet au contexte Spring. Ajoutons un bean d'une classe Parrot que nous avons définie également un **String** et un **Integer** et voyons que cela fonctionne.

Créez un sous-package beans_classesAPI du package javabasedconf et lui ajoutons les classes Parrot, ProjectConfig et main définies ci-dessous .



Classe Parrot.java

```
package com.spring.core.javabasedconf.beans_classesAPI;

public class Parrot {
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

Classe ProjectConfig.java

```
package com.spring.core.javabasedconf.beans_classesAPI;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class ProjectConfig {
    @Bean
    Parrot parrot() {
        var p = new Parrot();
        p.setName("Koko");
    }
}
```

```

        return p;
    }

    @Bean
    String hello() {
        return "Hello";
    }

    @Bean
    Integer ten() {
        return 10;
    }
}

```

Classe Main.java

```

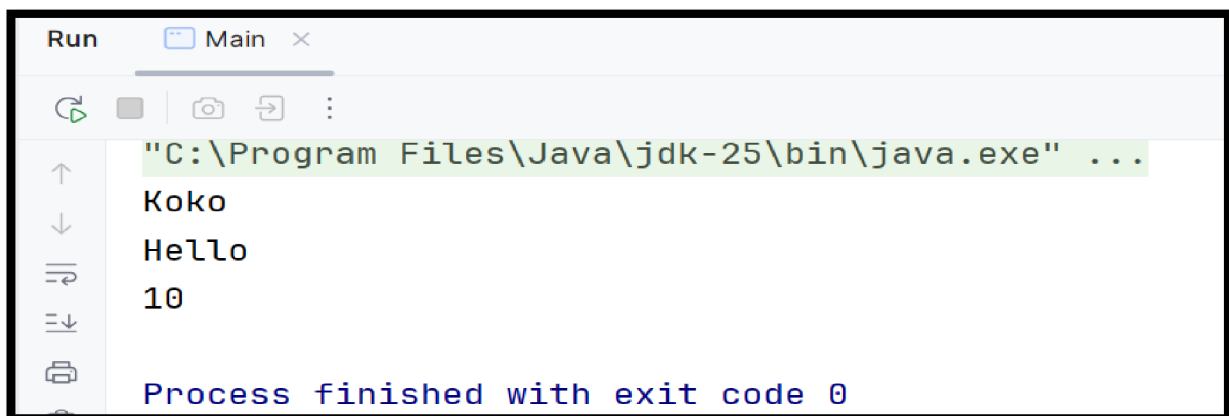
package com.spring.core.javabasedconf.beans_classesAPI;

import
org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {
        var context = new
AnnotationConfigApplicationContext(ProjectConfig.class);
        Parrot p = context.getBean(Parrot.class);
        System.out.println(p.getName());
        String s = context.getBean(String.class);
        System.out.println(s);
        Integer n = context.getBean(Integer.class);
        System.out.println(n);
    }
}

```

Résultat d'exécution du programme



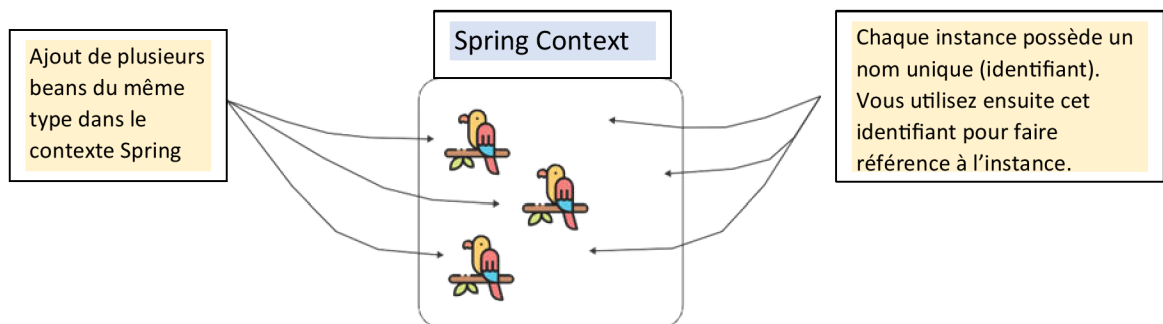
```

Run    Main x
[C] [X] [O] [F] [V]
"C:\Program Files\Java\jdk-25\bin\java.exe" ...
Koko
Hello
10
Process finished with exit code 0

```

Définition de plusieurs beans d'une même classe : NoUniqueBeanDefinitionException

Jusqu'à présent, nous avons ajouté un ou plusieurs beans de types différents au contexte Spring. Mais pouvons-nous ajouter plus d'un objet du même type (figure c--dessous) ? Si oui, comment pouvons-nous faire référence individuellement à ces objets ?

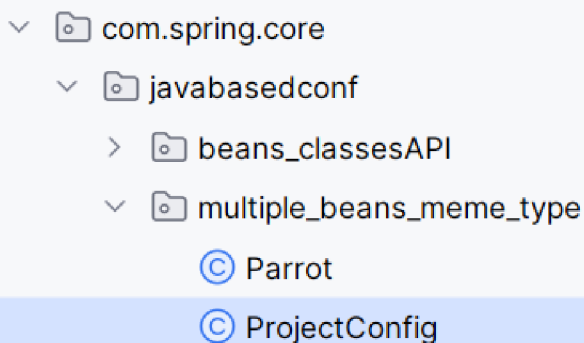


Vous pouvez ajouter plusieurs beans du même type au contexte Spring en utilisant plusieurs méthodes annotées avec **@Bean**. Chaque instance aura un identifiant unique. Pour y faire référence par la suite, vous devrez utiliser les identifiants des beans.

Vous pouvez ajouter plusieurs beans du même type au contexte Spring en utilisant plusieurs méthodes annotées avec **@Bean**. Chaque instance aura un identifiant unique. Pour y faire référence par la suite, vous devrez utiliser les identifiants des beans.

Le code qui suit montre comment déclarer trois beans de type **Parrot** dans la classe de configuration.

Commencer par créer un sous package `multiple_beans_meme_type`, puis créer les classes `Parrot` et la classe `ProjectConfig` qui crée 3 beans de type `Parrot`.



Classe `Parrot.java` (à copier pour la clarté du code)

```
package com.spring.core.javabasedconf.multiple_beans_meme_type;

public class Parrot {
    private String name;

    public String getName() {
```

```

        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

Classe ProjectConfig.java

```

package com.spring.core.javabasedconf.multiple_beans_meme_type;

import org.springframework.context.annotation.Bean;

public class ProjectConfig {
    @Bean
    Parrot parrot1() {
        var p = new Parrot();
        p.setName("Koko");
        return p;
    }

    @Bean
    Parrot parrot2() {
        var p = new Parrot();
        p.setName("Miki");
        return p;
    }

    @Bean
    Parrot parrot3() {
        var p = new Parrot();
        p.setName("Riki");
        return p;
    }
}

```

Bien sûr, vous ne pouvez plus récupérer les beans du contexte en spécifiant uniquement le type. Si vous le faites, vous obtiendrez une exception, car Spring ne peut pas deviner à quelle instance déclarée vous faites référence.

L'exécution du code de la classe Main définie ci-dessous provoque une exception (**NoUniqueBeanDefinition**) dans laquelle Spring vous indique que vous devez être précis quant à l'instance que vous souhaitez utiliser.

Classe Main.java

```

package com.spring.core.javabasedconf.multiple_beans_meme_type;

import

```

```

org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {
        var context = new
AnnotationConfigApplicationContext(ProjectConfig.class);
        /*L'instruction ci-dessous cause une exception
        * car Spring ne peut pas deviner à laquelle des
        * trois instances de **Parrot** vous faites référence.
        */
        Parrot p = context.getBean(Parrot.class);
        System.out.println(p.getName());
    }
}

```

L'exécution du code cause l'exception ci-dessous :

```

"C:\Program Files\Java\jdk-25\bin\java.exe" ....

Exception in thread "main" org.springframework.beans.factory.NoUniqueBeanDefinitionException: No qualifying bean of type
'com.spring.core.javabasedconf.multiple_beans_meme_type.Parrot' available: expected single matching bean but found 3: parrot1,parrot2,parrot3

    at org.springframework.beans.factory.support.DefaultListableBeanFactory.resolveNamedBean(DefaultListableBeanFactory.java:1612)
    at org.springframework.beans.factory.support.DefaultListableBeanFactory.resolveBean(DefaultListableBeanFactory.java:556)
    at org.springframework.beans.factory.support.DefaultListableBeanFactory.getBean(DefaultListableBeanFactory.java:384)
    at org.springframework.beans.factory.support.DefaultListableBeanFactory.getBean(DefaultListableBeanFactory.java:377)
    at org.springframework.context.support.AbstractApplicationContext.getBean(AbstractApplicationContext.java:1317)
    at com.spring.core.javabasedconf.multiple_beans_meme_type.Main.main(Main.java:12)

```

Pour résoudre ce problème d'ambiguïté, vous devez faire référence précisément à l'une des instances en utilisant le nom du bean. Par défaut, Spring utilise les noms des méthodes annotées avec **@Bean** comme noms des beans eux-mêmes.

Dans notre cas, les beans portent les noms **parrot1**, **parrot2** et **parrot3** (rappelez-vous, la méthode représente le bean). Vous pouvez retrouver ces noms dans l'extrait de code précédent, dans le message de l'exception.

Pour éviter l'exception `NoUniqueBeanDefinitionException`, il faut utiliser la signature de la méthode `getBeans` suivante: **context.getBean (<Bean Name>, <Bean Type>);**

Modifions maintenant la méthode **main** pour faire explicitement référence à l'un de ces beans en utilisant son nom. Le code ci-dessous fait référence au bean **parrot2**.

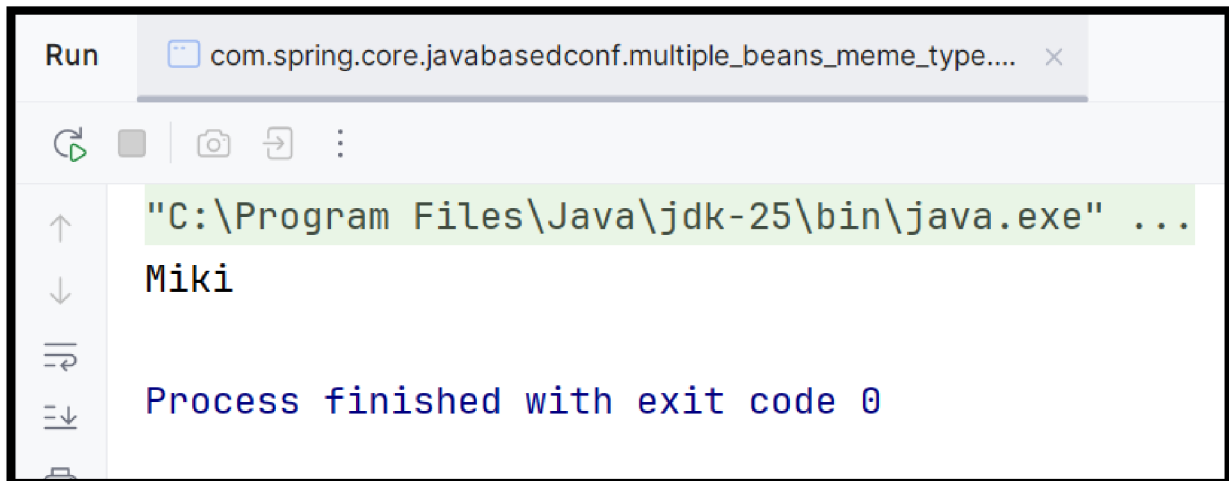
Classe Main.java

```
package com.spring.core.javabasedconf.multiple_beans_meme_type;

import
org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {
        var context = new
AnnotationConfigApplicationContext(ProjectConfig.class);
        /*L'instruction ci-dessous cause l'exception
NoUniqueBeanDefinitionException
* car Spring ne peut pas deviner à laquelle des
* trois instances de **Parrot** vous faites référence.
*/
        //Parrot p = context.getBean(Parrot.class);
        /*Dans l'instruction ci-dessous, le 1er paramètre fait référence
*an nom du bean
*/
        Parrot p = context.getBean("parrot2", Parrot.class);
        System.out.println(p.getName());
    }
}
```

En exécutant l'application maintenant, vous n'obtiendrez plus d'exception. À la place, vous verrez dans la console le nom du deuxième **parrot**, Miki.

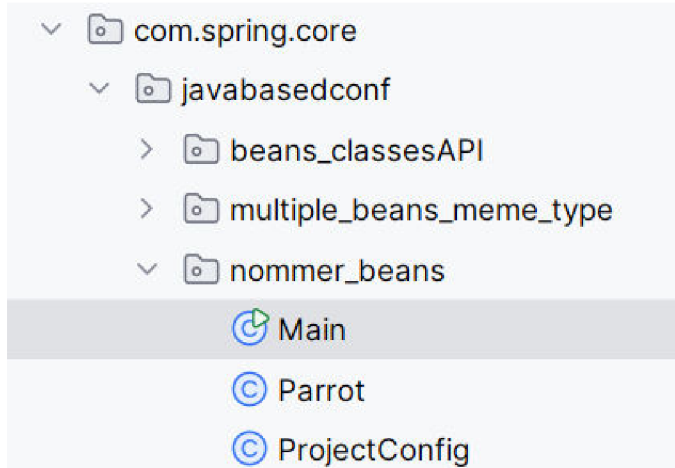


Nommer les beans

Pour attribuer le nom à un bean autre que celui du nom de la méthode qui le crée, on peut utiliser l'une des syntaxes ci-dessous :

- `@Bean(name = "miki")`
- `@Bean(value = "miki")`
- `@Bean("miki")`

Créer le sous_package **nommer_beans** et lui ajouter les classes Parrot, ProjectConfig, et Main ci-dessous,



Classe Parrot.java (à copier pour la clarté du code)

```
package com.spring.core.javabasedconf.nommer_beans;

public class Parrot {
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

Classe ProjectConfig.java

```
package com.spring.core.javabasedconf.nommer_beans;

import org.springframework.context.annotation.Bean;
```

```

public class ProjectConfig {
    @Bean (name="koko_bean") //Nom du bean
    Parrot parrot1() {
        var p = new Parrot();
        p.setName("Koko"); //Nom du Parrot
        return p;
    }

    @Bean(value="miki_bean")
    Parrot parrot2() {
        var p = new Parrot();
        p.setName("Miki");
        return p;
    }

    @Bean("riki_bean")
    Parrot parrot3() {
        var p = new Parrot();
        p.setName("Riki");
        return p;
    }
}

```

```

package com.spring.core.javabasedconf.nommer_beans;

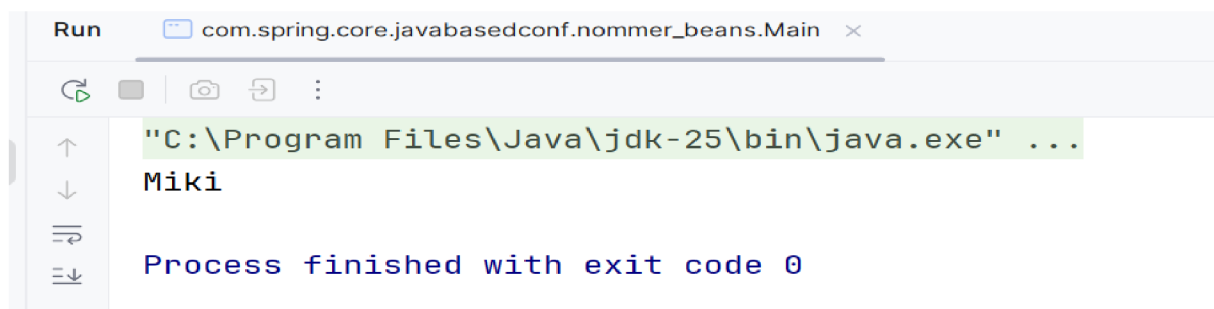
import
org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {
        var context = new
AnnotationConfigApplicationContext(ProjectConfig.class);

        Parrot p = context.getBean("miki_bean", Parrot.class);
        System.out.println(p.getName());
    }
}

```

En exécutant le programme, nous obtenons le résultat ci-dessous :



```

Run com.spring.core.javabasedconf.nommer_beans.Main x
"C:\Program Files\Java\jdk-25\bin\java.exe" ...
Miki
Process finished with exit code 0

```


Définir un bean comme principal avec l'annotation @Primary

Il existe une autre option pour référencer des beans dans le contexte lorsque vous avez plusieurs instances du même type.

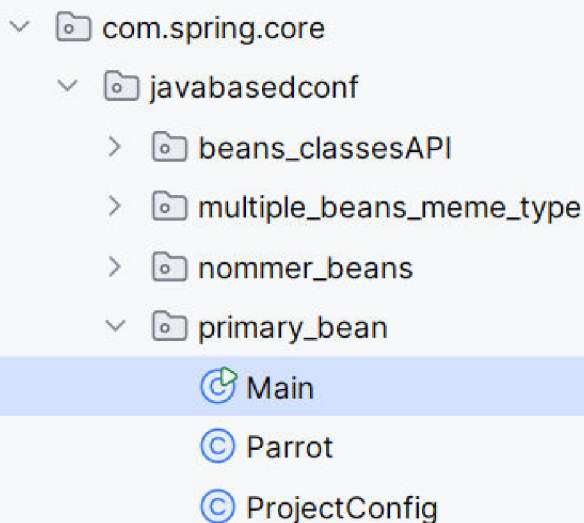
Lorsque vous avez plusieurs beans du même type dans le contexte Spring, vous pouvez en définir un comme principal. Vous marquez le bean que vous souhaitez définir comme principal à l'aide de l'annotation **@Primary**.

Un bean principal est celui que Spring choisira s'il dispose de plusieurs options et que vous ne spécifiez pas de nom ; le bean principal est simplement le choix par défaut de Spring.

L'extrait de code suivant montre à quoi ressemble une méthode **@Bean** annotée avec **@Primary**:

```
@Bean
@Primary
Parrot parrot2() {
    var p = new Parrot();
    p.setName("Miki"); return
    p;
}
```

Si vous faites référence à un **Parrot** sans spécifier le nom du bean , Spring sélectionnera désormais **Miki** par défaut. Bien sûr, vous ne pouvez définir qu'un seul bean d'un type donné comme principal.



Classe Parrot.java (à copier pour la clarté du code)

```
package com.spring.core.javabasedconf.nommer_beans;

public class Parrot {
    private String name;

    public String getName() {
        return name;
    }
}
```

```

    }

    public void setName(String name) {
        this.name = name;
    }
}

```

Classe ProjectConfig.java

```

package com.spring.core.javabasedconf.primary_bean;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Primary;

public class ProjectConfig {

    @Bean
    Parrot parrot1() {
        var p = new Parrot();
        p.setName("Koko"); //Nom du Parrot
        return p;
    }

    @Bean
    @Primary //Définition du bean principal
    Parrot parrot2() {
        var p = new Parrot();
        p.setName("Miki");
        return p;
    }

    @Bean
    Parrot parrot3() {
        var p = new Parrot();
        p.setName("Riki");
        return p;
    }
}

```

Classe Main.java

```

package com.spring.core.javabasedconf.primary_bean;

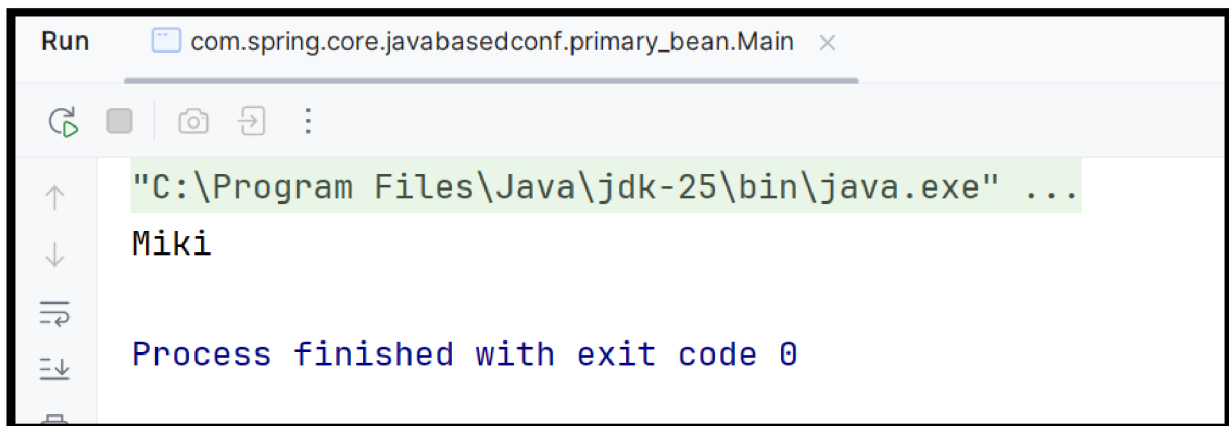
import
org.springframework.context.annotation.AnnotationConfigApplic
ationContext;

public class Main {
    public static void main(String[] args) {
        var context = new

```

```
AnnotationConfigApplicationContext (ProjectConfig.class);  
  
    Parrot p = context.getBean(Parrot.class);  
    System.out.println(p.getName());  
  
}
```

Résultat d'exécution



```
Run com.spring.core.javabasedconf.primary_bean.Main x  
"C:\Program Files\Java\jdk-25\bin\java.exe" ...  
Miki  
Process finished with exit code 0
```